Project Documentation

Date	**02 October 2026**
Team ID	**SWTID-2026-3953**
Project Name	**Implement Client Script & UI Policy (Incident)**
Maximum Marks	**4 Marks**

Project Description: This micro project focuses on implementing **Client Scripts and UI Policies** in the ServiceNow Incident module to dynamically control the behavior and appearance of incident form fields.

The Client Script is used to execute client-side JavaScript when specific form events occur, such as when an Incident form is loaded, submitted, or when a field value changes. The UI Policy is used to dynamically make fields **mandatory, visible, or read-only** based on predefined conditions.

The project demonstrates how Incident form behavior can be customized without modifying the underlying database structure. When an Incident record meets the configured condition, the Client Script and UI Policy automatically apply the required form behavior.

For example, when the **Incident State** or **Priority** meets a specified condition, the system can dynamically control fields such as **Description, Assignment Group, Assigned To, or Resolution Notes**. This helps users enter the required information and improves data consistency.

The project also includes testing of the configured Client Script and UI Policy by creating and updating Incident records and verifying that the expected field behavior is applied

Scenarios

• **Scenario 1:** An incident user opens a new Incident form and enters the required incident information.

• **Scenario 2:** When the configured Incident condition is satisfied, the UI Policy dynamically changes the visibility, mandatory status, or read-only status of the selected fields.

• **Scenario 3:** When the specified field value changes, the Client Script executes the configured client-side logic and updates the Incident form accordingly.

• **Scenario 4:** The user submits or updates the Incident after verifying that the configured field rules are correctly applied.

Technical Architecture:

Description: The solution uses a **ServiceNow Incident Form Customization Architecture**.

The Incident form acts as the input layer where users enter incident information. The configured **UI Policy** evaluates the specified condition and dynamically changes the field properties such as **Mandatory, Visible, and Read-only**.

The **Client Script** executes client-side JavaScript based on the configured form event and condition. It can be used to dynamically set field values, display messages, validate information, or control form behavior.

The overall flow is:

User → Incident Form → Field Event/Condition → Client Script / UI Policy → Dynamic Field Behavior → Incident Record

The complete configuration and execution are performed within the ServiceNow development instance.

.

Pre-requisites:

• **ServiceNow Development Instance:** Access to create and configure Client Scripts, UI Policies, and Incident records.

• **Incident Module Access:** Permission to create, view, and update Incident records.

• **Client Script Knowledge:** Basic understanding of client-side JavaScript and ServiceNow form events.

• **UI Policy Knowledge:** Basic understanding of UI Policy conditions and field actions.

• **Incident Form Knowledge:** Understanding of commonly used Incident fields such as Incident Number, Caller, Short Description, Description, State, Priority, Assignment Group, and Assigned To.

• **Testing Knowledge:** Ability to test Incident form behavior using different field values and conditions.

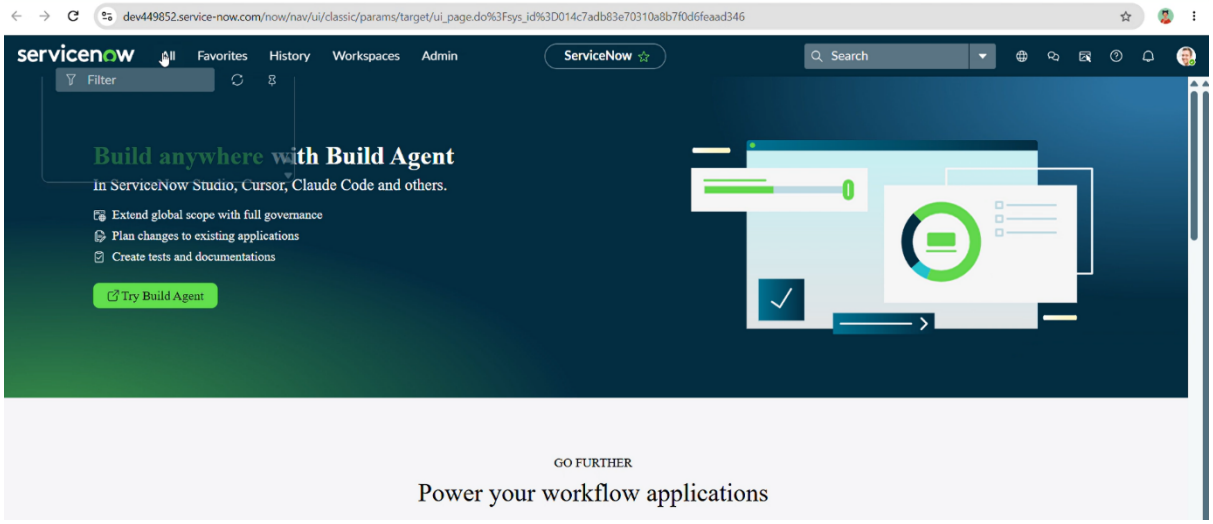
Project Workflow:

Milestone 1: Incident Form Analysis and Requirement Identification

• Activity 1.1: Open the ServiceNow Incident module and review the Incident form.

• Activity 1.2: Identify the fields that require dynamic behavior.

• Activity 1.3: Define the conditions under which the selected fields should become mandatory, visible, or read-only.



Milestone 2: Creation of UI Policy

• Activity 2.1: Navigate to the UI Policies section through the ServiceNow application menu.

• Activity 2.2: Create a new UI Policy for the Incident table.

• Activity 2.3: Configure the required condition for the UI Policy.

• Activity 2.4: Create UI Policy Actions for the required Incident fields.

• Activity 2.5: Configure the field properties such as Mandatory, Visible, and Read-only.

Milestone 3: Creation of Client Script

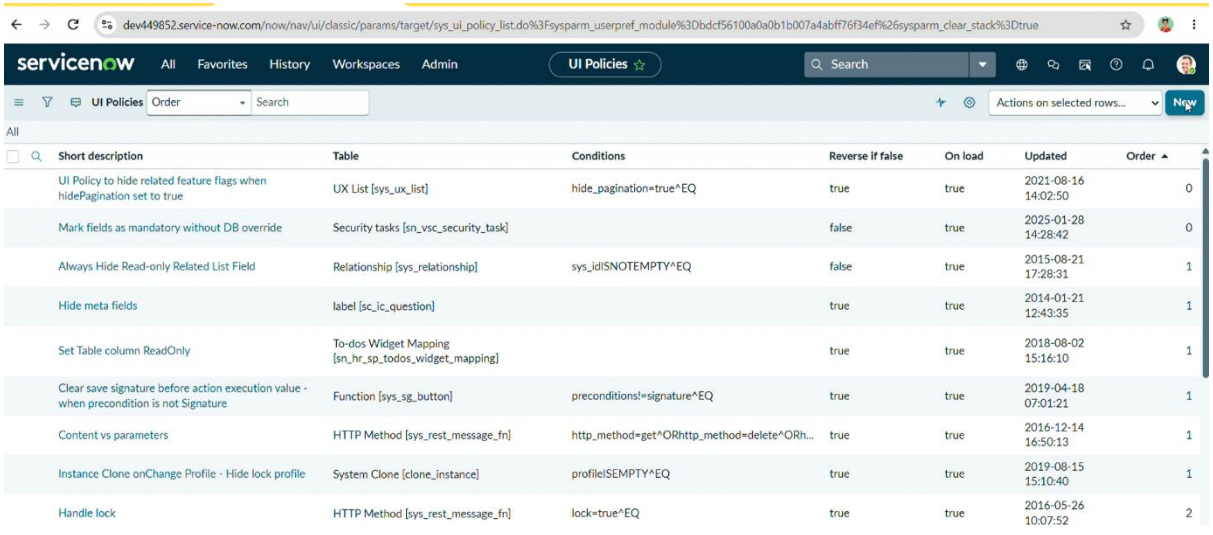
• Activity 3.1: Navigate to Client Scripts through the ServiceNow application menu.

• Activity 3.2: Create a Client Script for the Incident table.

• Activity 3.3: Select the appropriate Client Script type such as onLoad, onChange, onSubmit, or onCellEdit, depending on the requirement.

• Activity 3.4: Configure the required field and condition.

• Activity 3.5: Add the required JavaScript logic to control the Incident form behavior.



Milestone 4: Testing and Validation

• Activity 4.1: Open a new Incident record.

• Activity 4.2: Enter values that satisfy the configured UI Policy condition.

• Activity 4.3: Verify whether the selected fields become mandatory, visible, or read-only as configured.

• Activity 4.4: Change the relevant field value and verify the Client Script behavior.

• Activity 4.5: Submit and update the Incident record and verify that the configuration works correctly.

Milestone 1: Incident Form Analysis and Requirement Identification

This milestone identifies the Incident fields and form behavior required for the project. The Incident form provides the input interface where users enter and modify incident information.

The project focuses on improving the form dynamically using Client Scripts and UI Policies rather than manually controlling the fields.

Activity 1.1: Analyze Incident Form

• Open the All menu in ServiceNow.

• Search for Incidents.

• Open an existing Incident or create a new Incident.

• Review the available Incident fields.

Activity 1.2: Identify Required Fields

• Select the fields that require dynamic behavior.

• Identify the condition that should trigger the behavior.

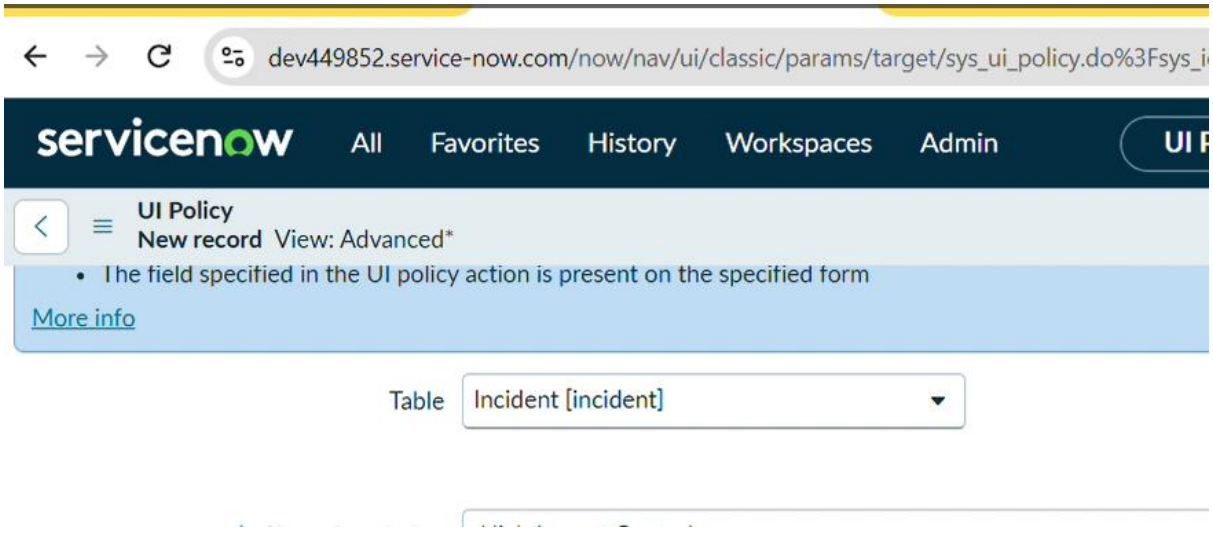
• Decide whether the field should become mandatory, visible, or read-only.

Activity 1.3: Define Form Behavior

The required form behavior is documented before creating the configuration.

Example:

Condition → Incident field changes → UI Policy/Client Script executes → Expected form behavior



Milestone 2: Creation of UI Policy

This milestone creates the UI Policy required to dynamically control Incident form fields.

Activity 2.1: Create UI Policy

• Open All.

• Search for UI Policies.

• Open the UI Policies module.

• Click New.

• Select the Incident table.

• Enter the required UI Policy name.

Activity 2.2: Configure Condition

• Define the condition based on the selected Incident field.

• Select the required operator and value.

• Enable the UI Policy.

Activity 2.3: Configure UI Policy Actions

• Open the UI Policy Actions section.

• Select the required Incident field.

• Configure the field behavior.

The field can be configured as:

- Mandatory
- Visible
- Read-only

The UI Policy dynamically applies these properties whenever the configured condition is satisfied.

Milestone 3: Creation of Client Script

This milestone implements the Client Script required for client-side Incident form behavior.

Activity 3.1: Create Client Script

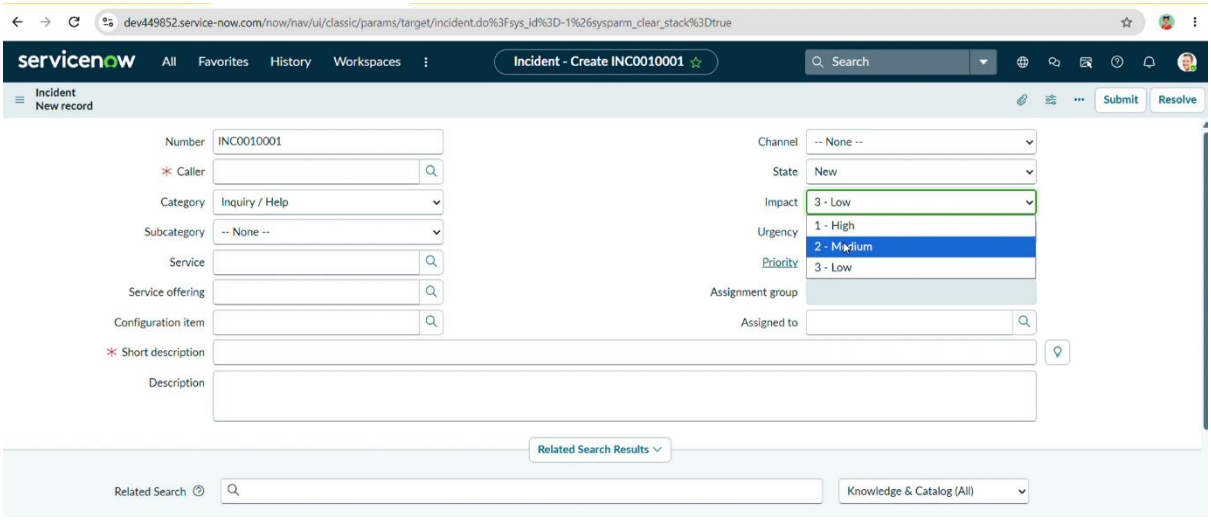
• Open All.

• Search for Client Scripts.

• Click New.

• Select the Incident table.

• Enter the Client Script name.



Activity 3.2: Select Client Script Type

The appropriate Client Script type is selected based on the requirement.

Possible types include:

• **onLoad:** Executes when the Incident form loads.

• **onChange:** Executes when the selected field value changes**.**

• **onSubmit:** Executes when the user submits the form.

• **onCellEdit:** Executes when a list cell is edited.

Activity 3.3: Configure Client Script Logic

The Client Script contains client-side JavaScript that controls the required Incident form behavior.

The script can be used to:

• **Set field values.**

• **Make fields mandatory or non-mandatory.**

• **Hide or display fields.**

• **Make fields read-only or editable.**

• **Display information or error messages.**

• **Validate Incident information before submission.**

Milestone 4: Testing and Validation

This milestone verifies whether the Client Script and UI Policy work correctly on the Incident form.

Activity 4.1: Test UI Policy

• Open the Incident form.

• Enter the condition required by the UI Policy.

• Verify the selected field behavior.

• Check whether the field becomes mandatory, visible, or read-only as configured.

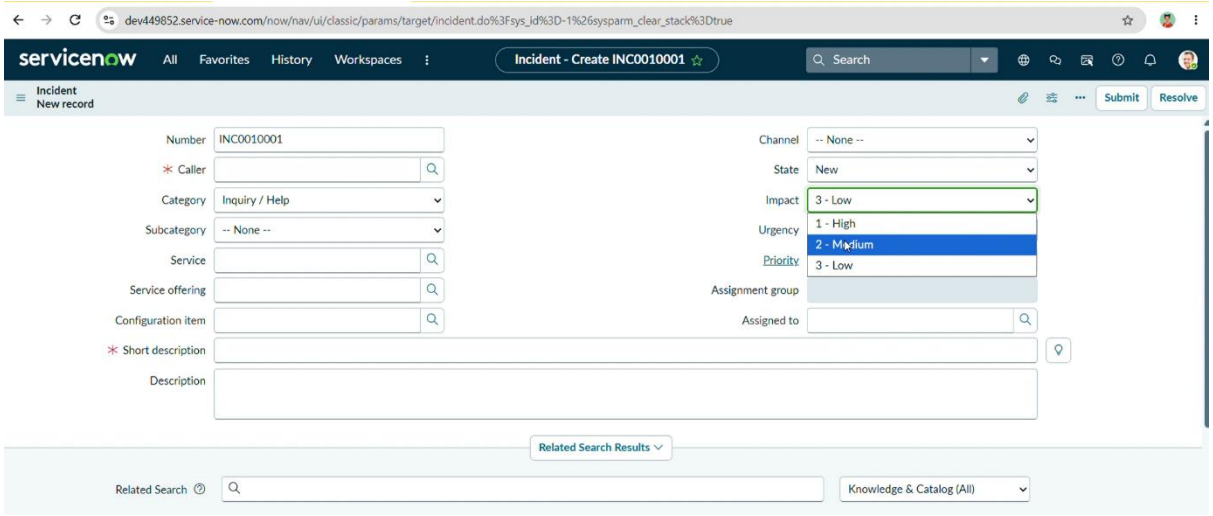
Activity 4.2: Test Client Script

• Open or create an Incident.

• Change the field used by the Client Script.

• Verify that the script executes correctly.

• Check whether the expected field value or form behavior is applied.



Activity 4.3: Validate Incident Submission

• Enter the required information.

• Submit or update the Incident.

• Verify that the Incident is successfully saved.

• Confirm that the configured Client Script and UI Policy do not produce unexpected behavior.

Validation Summary

• **Incident Form Tested:** Yes.
• **UI Policy Condition Tested:** Yes.
• **UI Policy Action Verified:** Yes.
• **Client Script Execution Tested:** Yes.
• **Dynamic Field Behavior Verified:** Yes.
• **Incident Submission Tested:** Yes.
• **Configuration Errors:** No errors observed during testing.

The final submission should be supported with screenshots showing the **Client Script configuration, UI Policy configuration, UI Policy Actions, Incident form, and the resulting dynamic field behavior**.**Milestone 4: Creation Of reports & dashboards**

This milestone demonstrates the end-to-end use of the completed import configuration with new employee data in Excel format.

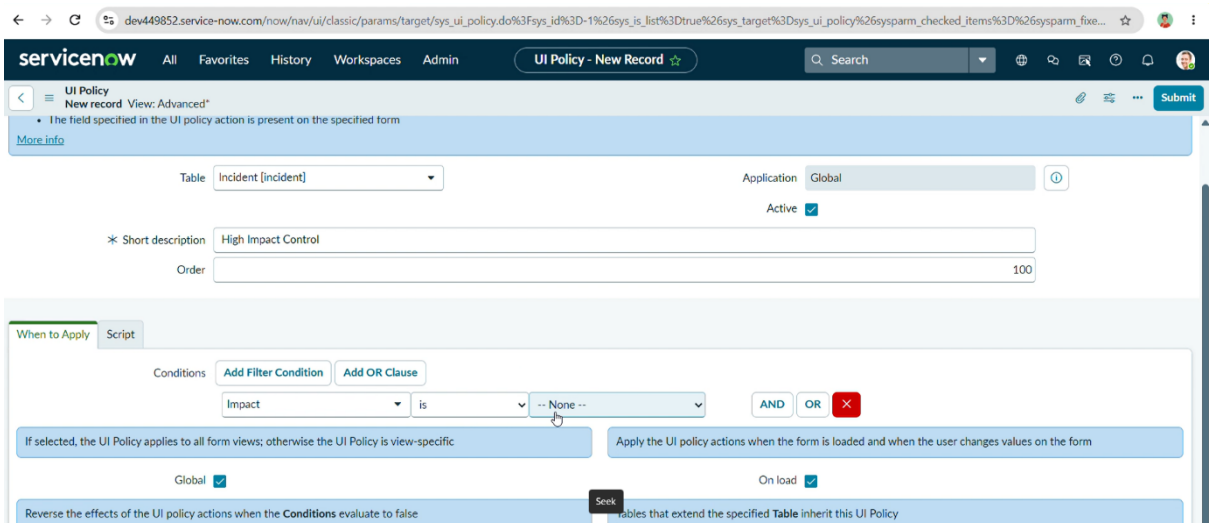
Activity 4.1: Test UI Policy

• Open the Incident form.

• Enter the condition required by the UI Policy.

• Verify the selected field behavior.

• Check whether the field becomes mandatory, visible, or read-only as configured



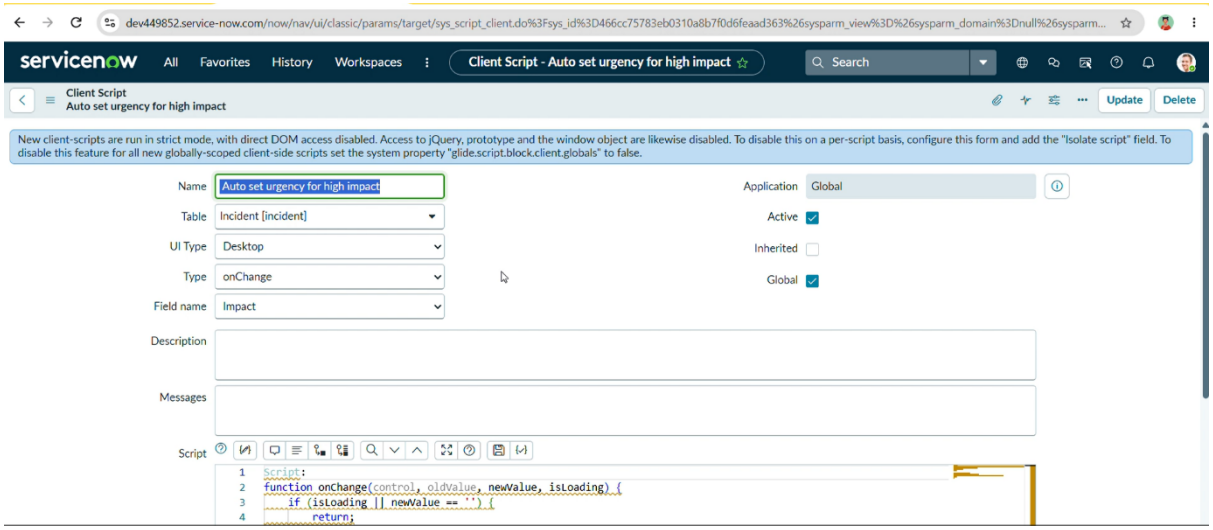
Activity 4.2: Test Client Script

• Open or create an Incident.

• Change the field used by the Client Script.

• Verify that the script executes correctly.

• Check whether the expected field value or form behavior is applied.



Project Repository and Submission Structure

The project deliverables are organized according to the project submission structure.

Implement-Client-Script-UI-Policy-Incident

├── 1. Brainstorming & Ideation

├── 2. Requirement Analysis

├── 3. Project Design Phase

├── 4. Project Planning Phase

├── 5. Project Development Phase

├── 6. Project Testing

├── 7. Project Documentation

└── 8. Project Demonstration

.

Project Artifacts

• Incident form requirement analysis.

• UI Policy configuration.

• UI Policy Action configuration.

• Client Script configuration.

• Client-side JavaScript logic.

• Incident testing records.

• Validation screenshots.

• Final project documentation.

• Project demonstration video/screenshots.

Exploring the ServiceNow Project PagesIncident Page

Description:
The Incident page represents the main form used for creating and updating Incident records. The configured Client Script and UI Policy operate on this form**.**

UI Policy Page

Description:
The UI Policy page contains the conditions that control the dynamic behavior of Incident fields. UI Policy Actions define whether the selected fields are mandatory, visible, or read-only.

Client Script Page

Description:
The Client Script page contains the client-side JavaScript configuration. It defines when the script executes and what action should be performed on the Incident form.

Incident Result Page

Description:
The Incident result page is used to verify the final behavior of the configuration. The user can confirm that the required fields and dynamic actions are working correctly.

Import Set / Staging Page:

Description: The Import Set Table page represents the staging layer of the project. It is where employee spreadsheet rows are held before transformation. The page should be used to review the source values and confirm that the expected rows were loaded.

Transform Map Page:

Description: The Transform Map page defines how source employee fields are mapped to the Employee Test target table. It is also where the Employee ID field map is configured for Coalesce so matching records can be updated.

Employee Test Target Page:

Description: The Employee Test table is the destination for the transformed records. After the transform runs, this page is used to verify that the target values match the source spreadsheet and that Coalesce behavior is working as intended.

Reports / Dashboard Page:

Description: The reporting layer presents the final employee data through Employees by Department, Employees by Location, and Employee List Report. These three reports are added to the Employee Analytics Dashboards dashboard for a consolidated view of the imported employee data.**Project Results and Verification The completed workflow demonstrates the dynamic customization of the ServiceNow Incident form using Client Scripts and UI Policies.**

End-to-End Result

- **Input: Incident information entered by the user.**

- **Condition: Configured Incident field condition.**

- **UI Policy: Dynamically controls field properties.**

- **Client Script: Executes client-side JavaScript based on the configured event.**

- **Form Behavior:** Fields dynamically become mandatory, visible, read-only, or editable according to the configuration**.**

- **Final Output:** Incident record is successfully created or updated with the required information.

Conclusion:

This project successfully implemented **Client Script and UI Policy functionality in the ServiceNow Incident module**. The configuration provides dynamic control over Incident form fields based on predefined conditions.
The UI Policy improves the Incident form by automatically controlling field properties such as **Mandatory, Visible, and Read-only**. This reduces manual effort and helps users provide the required information.
The Client Script further enhances the Incident form by executing client-side JavaScript when the configured form event occurs. It can perform field validation, update values, display messages, and dynamically control form behavior.
The combination of Client Script and UI Policy provides an effective way to customize the ServiceNow Incident form while maintaining a simple and user-friendly interface. Testing confirms that the configured behavior is applied when the specified conditions and events occur.
Overall, the project demonstrates how ServiceNow client-side configuration can improve **Incident data entry, validation, consistency, and user experience**.